

# LAPORAN PRAKTIKUM

## STRUKTUR DATA

“Algoritma Sorting”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

## **KATA PENGANTAR**

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul “Algoritma Sorting”

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 21 Mei 2026

Dennis Shauqi Akbar

## DAFTAR ISI

|                                         |     |
|-----------------------------------------|-----|
| KATA PENGANTAR .....                    | i   |
| DAFTAR ISI .....                        | ii  |
| BAB I PENDAHULUAN .....                 | 1   |
| 1.1 Latar Belakang .....                | 1   |
| 1.2 Tujuan.....                         | 1   |
| 1.3 Manfaat.....                        | 1   |
| BAB II PEMBAHASAN .....                 | 3   |
| 2.1 Pengertian Algoritma Sorting.....   | 3   |
| 2.2 Implementasi Pada Java .....        | 3   |
| 2.2.1 BubbleSort_2511533020 .....       | 3   |
| 2.2.2 InsertionSort_2511533020 .....    | 4   |
| 2.2.3 SelectionSort_2511533020.....     | 6   |
| 2.2.4 InsertionSortGUI_2511533020 ..... | 7   |
| BAB III PENUTUP.....                    | 10  |
| 3.1 Kesimpulan.....                     | 10  |
| 3.2 Saran.....                          | 10  |
| 3.3 Penutup.....                        | 10  |
| DAFTAR PUSTAKA.....                     | iii |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Sorting atau pengurutan data merupakan salah satu operasi yang paling sering dilakukan dalam pemrograman. Pada praktikum kali ini, saya mempelajari tiga algoritma sorting dasar yaitu Bubble Sort, Insertion Sort, dan Selection Sort. Ketiga algoritma ini termasuk dalam kategori comparison-based sorting karena proses pengurutannya dilakukan dengan membandingkan elemen-elemen data.

Bubble Sort bekerja dengan cara menukar elemen yang berdekatan jika urutannya salah, Insertion Sort bekerja dengan cara menyisipkan elemen ke posisi yang tepat, sedangkan Selection Sort bekerja dengan cara memilih elemen terkecil lalu menukarkannya ke posisi awal. Setiap algoritma memiliki kelebihan dan kekurangan masing-masing tergantung pada kondisi data yang akan diurutkan.

Selain membuat program berbasis console, pada praktikum ini saya juga membuat program GUI (Graphical User Interface) untuk memvisualisasikan proses Insertion Sort langkah demi langkah agar lebih mudah dipahami.

### **1.2 Tujuan**

Tujuan dari praktikum ini adalah memahami cara kerja ketiga algoritma sorting yaitu Bubble Sort, Insertion Sort, dan Selection Sort. Saya diharapkan mampu mengimplementasikan ketiga algoritma tersebut dalam bahasa Java, baik dalam bentuk console program maupun GUI. Selain itu, saya juga belajar membandingkan efisiensi ketiga algoritma berdasarkan jumlah iterasi dan pertukaran yang terjadi.

Pada program GUI Insertion Sort, saya juga belajar bagaimana menampilkan visualisasi proses sorting secara bertahap sehingga alur algoritma dapat diamati dengan jelas.

### **1.3 Manfaat**

Manfaat yang saya peroleh dari praktikum ini adalah pemahaman yang lebih mendalam tentang bagaimana data dapat diurutkan menggunakan berbagai metode. Saya jadi tahu bahwa tidak semua algoritma sorting cocok untuk semua kondisi data. Misalnya Bubble Sort cocok untuk data yang hampir terurut, Insertion Sort bagus untuk data kecil, sedangkan Selection Sort

memiliki jumlah pertukaran yang minimal. Program GUI dibuat juga berguna untuk membantu menjelaskan proses Insertion Sort kepada orang lain karena setiap langkah ditampilkan secara visual. Pengetahuan tentang sorting ini sangat berguna untuk pengembangan aplikasi yang membutuhkan pengurutan data seperti daftar nilai mahasiswa, daftar nama, atau daftar transaksi.

\Selain itu, praktikum ini melatih saya dalam melakukan debugging karena kesalahan pengaturan pointer pada DLL sering menyebabkan program error.

## BAB II

### PEMBAHASAN

#### 2.1 Pengertian Algoritma Sorting

Sorting adalah proses mengatur sekumpulan data ke dalam urutan tertentu, misalnya dari yang terkecil ke terbesar (ascending) atau dari terbesar ke terkecil (descending). Terdapat banyak algoritma sorting, tiga di antaranya adalah Bubble Sort, Insertion Sort, dan Selection Sort. Bubble Sort bekerja dengan cara membandingkan elemen yang berdekatan dan menukarnya jika tidak sesuai urutan. Proses ini diulang sebanyak  $n-1$  kali hingga semua elemen terurut. Insertion Sort bekerja seperti kita mengurutkan kartu di tangan, yaitu mengambil satu elemen lalu menyisipkannya ke posisi yang tepat di antara elemen yang sudah terurut.

Selection Sort bekerja dengan mencari elemen terkecil dari sisa data yang belum terurut, lalu menukarnya ke posisi paling depan. Ketiga algoritma ini memiliki kompleksitas waktu  $O(n^2)$  pada kasus terburuk, sehingga lebih cocok untuk data dalam jumlah kecil.

#### 2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

##### 2.2.1 BubbleSort\_2511533020

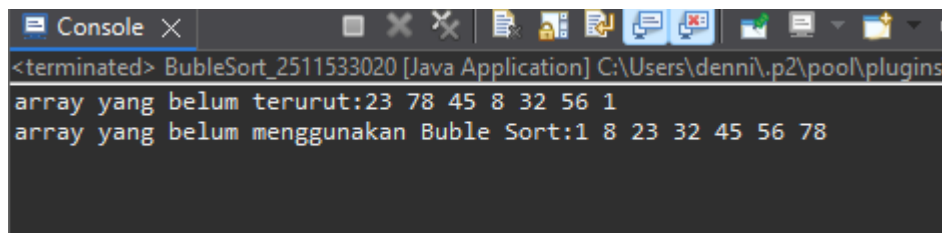
```
1 package pekan7_2511533020;
2
3 public class BubbleSort_2511533020 {
4     public static void bubbleSort (int[] arr) {
5         int n_3020 = arr.length;
6         for (int i_3020 = 0; i_3020 < n_3020; i_3020++) {
7             for (int j_3020 = 0; j_3020 < n_3020 - i_3020 - 1; j_3020++) {
8                 if (arr[j_3020] > arr[j_3020 + 1]) {
9                     int temp_3020 = arr[j_3020];
10                    arr[j_3020] = arr[j_3020 + 1];
11                    arr[j_3020 + 1] = temp_3020;
12                    // System.out.println("data: "+arr[j]+" "+arr[j+1]);
13                }
14            }
15        }
16    }
17    public static void main(String[] args) {
18        int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
19        int n_3020 = arr.length;
20        System.out.print("array yang belum terurut:");
21        for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
22            System.out.print(arr[i_3020] + " ");
23        System.out.println("");
24        bubbleSort(arr);
25        System.out.print("array yang belum menggunakan Buble Sort:");
26        for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
27            System.out.print(arr[i_3020] + " ");
28        System.out.println("");
29    }
30 }
31
32 }
33 }
```

(Gambar 2.2.1)

Program ini mengimplementasikan algoritma Bubble Sort untuk mengurutkan array bilangan bulat. Method bubbleSort menerima parameter array arr dan mengurutkannya secara ascending. Algoritma bekerja dengan dua perulangan bersarang. Perulangan luar (i\_3020) mengontrol jumlah pass sebanyak panjang array. Perulangan dalam (j\_3020) membandingkan

elemen `arr[j_3020]` dengan `arr[j_3020+1]`. Jika elemen kiri lebih besar dari elemen kanan, maka kedua elemen ditukar menggunakan variabel sementara `temp_3020`. Setelah setiap pass, elemen terbesar akan berpindah ke posisi paling kanan.

Pada method `main`, dibuat array berisi {23, 78, 45, 8, 32, 56, 1}. Array ditampilkan sebelum sorting, kemudian method `bubbleSort` dipanggil, lalu array ditampilkan kembali setelah sorting. Output yang dihasilkan menunjukkan array berubah dari [23, 78, 45, 8, 32, 56, 1] menjadi [1, 8, 23, 32, 45, 56, 78].



## 2.2.2 InsertionSort\_2511533020

```

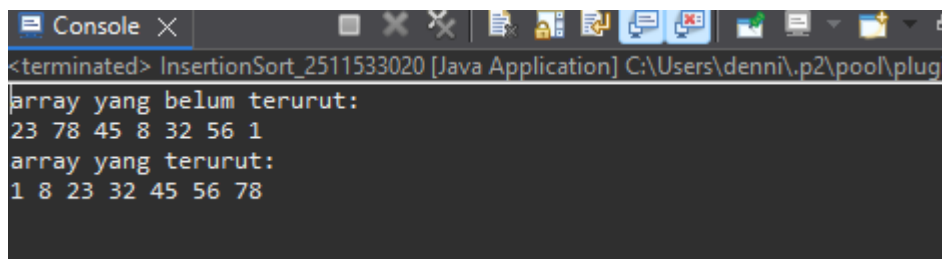
1 package pekan7_2511533020;
2
3 public class InsertionSort_2511533020 {
4     public static void insertionSort (int[] arr) {
5         int n_3020 = arr.length;
6         for (int i_3020 = 1; i_3020 < n_3020; i_3020++) {
7             int key_3020 = arr[i_3020];
8             int j_3020 = i_3020 - 1;
9             while (j_3020 >= 0 && arr[j_3020] > key_3020) {
10                 arr[j_3020 + 1] = arr[j_3020];
11                 j_3020--;
12             }
13             arr[j_3020 + 1] = key_3020;
14         }
15     }
16
17     public static void main(String[] args) {
18         int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
19         int n_3020 = arr.length;
20         System.out.printf("array yang belum terurut:\n");
21         for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
22             System.out.print(arr[i_3020] + " ");
23         System.out.println("");
24         insertionSort(arr);
25         for (int i_3020 = 0; i_3020 < n_3020 ; i_3020++)
26             System.out.print(arr[i_3020] + " ");
27         System.out.println("");
28
29
30
31
32
33
34     }
35 }
36

```

(Gambar 2.2.)

Program ini mengimplementasikan algoritma Insertion Sort untuk mengurutkan array bilangan bulat. Method insertionSort bekerja dengan membagi array menjadi dua bagian yaitu bagian yang sudah terurut (kiri) dan bagian yang belum terurut (kanan). Perulangan dimulai dari indeks 1 hingga akhir array. Untuk setiap  $i\_3020$ , nilai  $key\_3020$  diambil dari  $arr[i\_3020]$ , lalu variabel  $j\_3020$  diatur ke  $i\_3020-1$ . Selama  $j\_3020$  masih  $\geq 0$  dan  $arr[j\_3020]$  lebih besar dari  $key\_3020$ , maka  $arr[j\_3020+1]$  digeser ke kanan dan  $j\_3020$  dikurangi.

Setelah perulangan selesai,  $arr[j\_3020+1]$  diisi dengan  $key\_3020$ . Proses ini diulang hingga semua elemen terurut. Pada method main, array yang sama dengan Bubble Sort digunakan. Array ditampilkan sebelum dan setelah sorting. Hasil akhirnya sama yaitu array terurut secara ascending.



```
<terminated> InsertionSort_2511533020 [Java Application] C:\Users\denni\.p2\pool\plug
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

### 2.2.3 SelectionSort\_2511533020

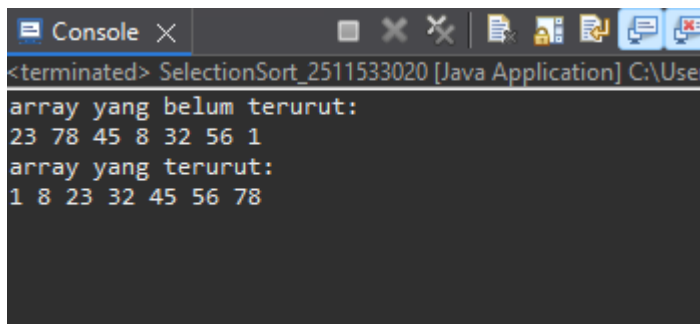
```
1 package pekan7_2511533020;
2
3 public class SelectionSort_2511533020 {
4     public static void selectionSort (int[] arr) {
5         int n_3020 = arr.length;
6         for (int i_3020 = 0; i_3020 < n_3020; i_3020++) {
7             int minIndex_3020 = i_3020;
8             for (int j_3020 = i_3020 + 1; j_3020 < n_3020; j_3020++) {
9                 if (arr[j_3020] < arr[minIndex_3020]) {
10                     minIndex_3020 = j_3020;
11                 }
12             }
13             int temp_3020 = arr[i_3020];
14             arr[i_3020] = arr[minIndex_3020];
15             arr[minIndex_3020] = temp_3020;
16         }
17     }
18
19     public static void main(String[] args) {
20         int arr[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n_3020 = arr.length;
22         System.out.printf("array yang belum terurut:\n");
23         for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
24             System.out.print(arr[i_3020] + " ");
25         System.out.println("");
26         selectionSort(arr);
27         System.out.printf("array yang terurut:\n");
28         for (int i_3020 = 0; i_3020 < n_3020; i_3020++)
29             System.out.print(arr[i_3020] + " ");
30         System.out.println("");
31     }
32 }
33
34 }
35
36 }
```

(Gambar 2.2.3)

Program ini mengimplementasikan algoritma Selection Sort. Method selectionSort bekerja dengan mencari nilai terkecil dari sisa array yang belum terurut, lalu menukarnya ke posisi yang tepat. Perulangan luar ( $i\_3020$ ) berjalan dari awal hingga akhir array. Untuk setiap

$i\_3020$ , variabel  $minIndex\_3020$  diatur ke  $i\_3020$ . Perulangan dalam ( $j\_3020$ ) berjalan dari  $i\_3020+1$  hingga akhir array. Jika ditemukan nilai  $arr[j\_3020]$  yang lebih kecil dari  $arr[minIndex\_3020]$ , maka  $minIndex\_3020$  diupdate. Setelah perulangan dalam selesai, nilai pada indeks  $i\_3020$  ditukar dengan nilai pada indeks  $minIndex\_3020$  menggunakan variabel sementara  $temp\_3020$ .

Dengan cara ini, elemen terkecil akan ditempatkan di posisi awal, kemudian elemen terkecil kedua di posisi kedua, dan seterusnya. Pada method main, array yang sama digunakan dan hasilnya juga sama yaitu array terurut.



```
<terminated> SelectionSort_2511533020 [Java Application] C:\User
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

## 2.2.4 InsertionSortGUI\_2511533020

```
1 package pekan7_2511533020;
2
3 import java.awt.BorderLayout;
4
5 public class InsertionSortGUI_2511533020 extends JFrame {
6     private static final long serialVersionUID = 1L;
7     private int[] array_3020;
8     private JLabel[] labelArray_3020;
9     private JButton stepButton_3020, resetButton_3020, setButton_3020;
10    private JTextField inputField_3020;
11    private JPanel panelArray_3020;
12    private JTextArea stepArea_3020;
13
14    private int i_3020 = 1, j_3020;
15    private boolean sorting_3020 = false;
16    private int stepCount_3020 = 1;
17
18    /**
19     * Launch the application.
20     */
21    /**
22     * Create the frame.
23     */
24    public InsertionSortGUI_2511533020() {
25        setTitle("Insertion Sort Langkah per Langkah");
26        setSize(750, 480);
27        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28        setLocationRelativeTo(null);
29        setLayout(new BorderLayout());
30
31        // Panel input
32        JPanel inputPanel = new JPanel(new FlowLayout());
33        inputField_3020 = new JTextField(30);
34        setButton_3020 = new JButton("Set Array");
35        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
36        inputPanel.add(inputField_3020);
37        inputPanel.add(setButton_3020);
38
39        panelArray_3020 = new JPanel();
40        panelArray_3020.setLayout(new FlowLayout());
41
42        JPanel controlPanel = new JPanel();
43        stepButton_3020 = new JButton("Langkah Selanjutnya");
44        resetButton_3020 = new JButton("Reset");
45        stepButton_3020.setEnabled(false);
46        controlPanel.add(stepButton_3020);
47        controlPanel.add(resetButton_3020);
48
49        stepArea_3020 = new JTextArea(5, 60);
50        stepArea_3020.setEditable(false);
51        stepArea_3020.setFont(new Font("Monospaced", Font.PLAIN, 14));
52        JScrollPane scrollPane = new JScrollPane(stepArea_3020);
53
54        add(inputPanel, BorderLayout.NORTH);
55        add(panelArray_3020, BorderLayout.CENTER);
56        add(controlPanel, BorderLayout.SOUTH);
57        add(scrollPane, BorderLayout.EAST);
58
59        setButton_3020.addActionListener(e -> setArrayFromInput_3020());
60        stepButton_3020.addActionListener(e -> performStep_3020());
61        resetButton_3020.addActionListener(e -> reset_3020());
62
63    }
64
65    private void setArrayFromInput_3020() {
66        String text = inputField_3020.getText().trim();
67        if (text.isEmpty()) return;
68        String[] parts = text.split(",");
69        array_3020 = new int[parts.length];
70        try {
71            for (int k = 0; k < parts.length; k++) {
72                array_3020[k] = Integer.parseInt(parts[k]);
73            }
74        } catch (NumberFormatException e) {
75            JOptionPane.showMessageDialog(this, "Masukkan angka yang valid");
76        }
77        labelArray_3020 = new JLabel[array_3020.length];
78        panelArray_3020.removeAll();
79        for (int k = 0; k < array_3020.length; k++) {
80            labelArray_3020[k] = new JLabel(array_3020[k]);
81            panelArray_3020.add(labelArray_3020[k]);
82        }
83        panelArray_3020.revalidate();
84        panelArray_3020.repaint();
85        stepArea_3020.setText("");
86        stepCount_3020 = 1;
87        stepButton_3020.setEnabled(true);
88    }
89
90    private void performStep_3020() {
91        if (!sorting_3020) {
92            sorting_3020 = true;
93            stepArea_3020.setText("Langkah 1: Menentukan elemen terkecil");
94            stepCount_3020++;
95        } else {
96            stepArea_3020.setText("Langkah 2: Memasukkan elemen terkecil ke posisi awal");
97            stepCount_3020++;
98        }
99        if (stepCount_3020 > 7) {
100            sorting_3020 = false;
101            stepArea_3020.setText("Array sudah terurut");
102            stepCount_3020 = 1;
103            stepButton_3020.setEnabled(false);
104        }
105    }
106
107    private void reset_3020() {
108        sorting_3020 = false;
109        stepArea_3020.setText("");
110        stepCount_3020 = 1;
111        stepButton_3020.setEnabled(true);
112    }
113}
```

```

11 array_3020[k] = Integer.parseInt(parts[k].trim());
12 }
13 } catch (NumberFormatException e) {
14 JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
15 + "dengan koma", "Error", JOptionPane.ERROR_MESSAGE);
16 return;
17 }
18 i_3020 = i;
19 stepCount_3020 = 1;
20 sorting_3020 = true;
21 stepButton_3020.setEnabled(true);
22 stepArea_3020.setText("");
23 panelArray_3020.removeAll();
24 labelArray_3020 = new JLabel[array_3020.length];
25 for (int k = 0; k < array_3020.length; k++) {
26 labelArray_3020[k] = new JLabel(String.valueOf(array_3020[k]));
27 labelArray_3020[k].setFont(new Font("Arial", Font.BOLD, 24));
28 labelArray_3020[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
29 labelArray_3020[k].setPreferredSize(new Dimension(50, 50));
30 labelArray_3020[k].setHorizontalAlignment(SwingConstants.CENTER);
31 panelArray_3020.add(labelArray_3020[k]);
32 }
33 panelArray_3020.revalidate();
34 panelArray_3020.repaint();
35 }
36 private void performStep_3020() {
37 if (i_3020 < array_3020.length && sorting_3020) {
38 int key = array_3020[i_3020];
39 int j_3020 = i_3020 - 1;
40
41 StringBuilder stepLog = new StringBuilder();
42 stepLog.append("Langkah ").append(stepCount_3020).
43 append(": Memasukkan ").append(key).append("\n");
44
45 while (j_3020 >= 0 && array_3020[j_3020] > key) {
46 array_3020[j_3020 + 1] = array_3020[j_3020];

```

```

127 j_3020--;
128 }
129 array_3020[j_3020 + 1] = key;
130
131 updateLabels();
132 stepLog.append("Hasil: ").append(arrayToString(array_3020)).append("\n\n");
133 stepArea_3020.append(stepLog.toString());
134
135 i_3020++;
136 stepCount_3020++;
137
138 if (i_3020 == array_3020.length) {
139 sorting_3020 = false;
140 stepButton_3020.setEnabled(false);
141 JOptionPane.showMessageDialog(this, "Sorting selesai!");
142 }
143
144 }
145 private void updateLabels() {
146 for (int k = 0; k < array_3020.length; k++) {
147 labelArray_3020[k].setText(String.valueOf(array_3020[k]));
148 }
149
150 }
151 private void reset_3020() {
152 inputField_3020.setText("");
153 panelArray_3020.removeAll();
154 panelArray_3020.revalidate();
155 panelArray_3020.repaint();
156 stepArea_3020.setText("");
157 stepButton_3020.setEnabled(false);
158 sorting_3020 = false;
159 i_3020 = 1;
160 stepCount_3020 = 1;
161 }
162

```

```

163 private String arrayToString(int[] arr_3020) {
164 StringBuilder sb = new StringBuilder();
165 for (int k = 0; k < arr_3020.length; k++) {
166 sb.append(arr_3020[k]);
167 if (k < arr_3020.length - 1) sb.append(", ");
168 }
169 return sb.toString();
170 }
171
172 public static void main(String[] args) {
173 SwingUtilities.invokeLater(() -> {
174 InsertionSortGUI_2511533820 gui = new InsertionSortGUI_2511533820();
175 gui.setVisible(true);
176 });
177 }
178

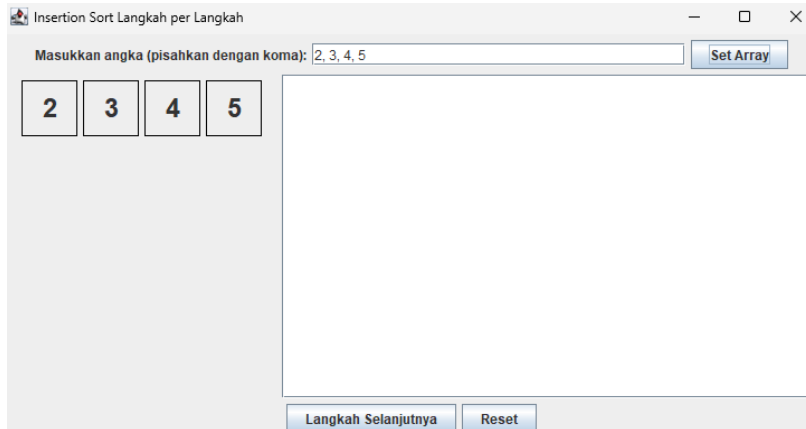
```

(Gambar 2.2.4)

Program ini adalah program GUI (Graphical User Interface) yang memvisualisasikan proses Insertion Sort langkah demi langkah. GUI dibuat menggunakan Java Swing. Program terdiri dari beberapa komponen: JTextField untuk memasukkan angka yang dipisahkan koma, JButton "Set Array" untuk memproses input, JButton "Langkah Selanjutnya" untuk menjalankan sorting satu langkah, JButton "Reset" untuk mengembalikan ke keadaan awal, JPanel untuk menampilkan array dalam bentuk label visual, dan JTextArea untuk menampilkan log setiap langkah. Saat pengguna mengklik "Set Array", input diurai menjadi array integer, lalu ditampilkan dalam bentuk kotak-kotak berisi angka. Tombol "Langkah Selanjutnya" akan

aktif. Setiap kali tombol ditekan, program menjalankan satu iterasi dari algoritma Insertion Sort (memasukkan satu elemen ke posisi yang tepat). Log langkah ditampilkan di JTextArea, dan label-label di panel diperbarui. Proses berlanjut hingga semua elemen terurut, lalu tombol dinonaktifkan dan muncul pesan "Sorting selesai!".

Program ini sangat membantu untuk memahami alur Insertion Sort secara visual



## **BAB III**

### **PENUTUP**

#### **3.1 Kesimpulan**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ketiga algoritma sorting (Bubble Sort, Insertion Sort, dan Selection Sort) berhasil diimplementasikan dalam bahasa Java. Ketiganya dapat mengurutkan array bilangan bulat dari nilai terkecil ke terbesar dengan hasil yang sama. Perbedaan utama terletak pada cara kerja dan jumlah perbandingan serta pertukaran yang dilakukan.

Bubble Sort melakukan banyak pertukaran, Insertion Sort efisien untuk data yang hampir terurut, sedangkan Selection Sort memiliki jumlah pertukaran yang minimal. Program GUI Insertion Sort yang dibuat juga berhasil menampilkan visualisasi proses sorting langkah demi langkah sehingga memudahkan pemahaman terhadap alur algoritma

#### **3.2 Saran**

Untuk pengembangan lebih lanjut, sebaiknya program Bubble Sort, Insertion Sort, dan Selection Sort dilengkapi dengan penghitung jumlah iterasi dan pertukaran agar dapat membandingkan efisiensi ketiga algoritma secara langsung. Pada program GUI Insertion Sort, sebaiknya ditambahkan fitur untuk memilih kecepatan langkah (delay) atau fitur sort otomatis tanpa harus klik tombol terus menerus.

Selain itu, ketiga algoritma ini dapat dikembangkan untuk mengurutkan data bertipe String (misalnya nama mahasiswa) menggunakan `compareToIgnoreCase()`. Secara umum, kode yang dibuat sudah rapi dan mengikuti aturan penamaan dengan akhiran NIM 3020

#### **3.3 Penutup**

Demikian laporan praktikum Algoritma Sorting ini saya buat. Laporan ini berisi penjelasan mengenai tiga algoritma sorting yaitu Bubble Sort, Insertion Sort, dan Selection Sort, serta implementasinya dalam bahasa Java baik versi console maupun GUI.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu kritik dan saran yang membangun sangat saya harapkan untuk perbaikan di masa mendatang.

## **DAFTAR PUSTAKA**

- [1] Oracle "Arrays (Java Platform SE 17)," Oracle. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html> (diakses: 21 Mei 2026).
- [2] W3Schools, "Java Sorting Algorithms," W3Schools. [Online]. Available: [https://www.w3schools.com/java/java\\_sorting.asp](https://www.w3schools.com/java/java_sorting.asp) (diakses: 21 Mei 2026).
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [4] Wikipedia, "Sorting algorithm," Wikipedia. [Online]. Available: [https://en.wikipedia.org/wiki/Sorting\\_algorithm](https://en.wikipedia.org/wiki/Sorting_algorithm) (diakses: 21 Mei 2026).

## TUGAS PRAKTIKUM

- **KELAS ADT**

```
package pekan7_2511533020;

class Mahasiswa_2511533020 {
    private String nama_3020;
    private String nim_3020;
    private String prodi_3020;

    // Constructor
    public Mahasiswa_2511533020(String nama_3020, String nim_3020, String
prodi_3020) {
        this.nama_3020 = nama_3020;
        this.nim_3020 = nim_3020;
        this.prodi_3020 = prodi_3020;
    }

    // Getter
    public String getNama_3020() {
        return nama_3020;
    }

    public String getNim_3020() {
        return nim_3020;
    }

    public String getProdi_3020() {
        return prodi_3020;
    }

    // Setter
    public void setNama_3020(String nama_3020) {
        this.nama_3020 = nama_3020;
    }

    public void setNim_3020(String nim_3020) {
        this.nim_3020 = nim_3020;
    }

    public void setProdi_3020(String prodi_3020) {
        this.prodi_3020 = prodi_3020;
    }

    // toString untuk menampilkan data
    @Override
    public String toString() {
        return nama_3020 + " (" + nim_3020 + " - " + prodi_3020 + ")";
    }
}
```

Kelas Mahasiswa\_2511533020 berisi tiga atribut yaitu nama\_3020, nim\_3020, dan prodi\_3020. Ketiga atribut ini bertipe String dan menggunakan akhiran 3020. Kelas ini memiliki constructor untuk mengisi ketiga atribut saat objek dibuat. Method getter (getNama\_3020(), getNim\_3020(),

getProdi\_3020()) disediakan untuk mengambil nilai dari masing-masing atribut. Method setter (setNama\_3020(), setNim\_3020(), setProdi\_3020()) disediakan untuk mengubah nilai atribut. Method toString() di-override sehingga ketika objek dicetak, outputnya berbentuk "nama (nim - prodi)".

- **KELAS DRIVER**

```
1 package pekan7_2511533020;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import java.awt.*;
6 import java.awt.event.ActionEvent;
7 import java.awt.event.ActionListener;
8 import java.util.ArrayList;
9
10
11 public class SortingGUI_2511533020 extends JFrame {
12     // Komponen GUI
13     private JTextField fieldNama_3020, fieldNim_3020, fieldProdi_3020;
14     private JButton btnTambah_3020, btnHapus_3020, btnSort_3020;
15     private JComboBox<String> comboAlgoritma_3020;
16     private JTable tabelMahasiswa_3020;
17     private DefaultTableModel tableModel_3020;
18     private JTextArea areaVisualisasi_3020;
19     private JScrollPane scrollTabel_3020, scrollVisual_3020;
20
21     // Tempat Simpan Data
22     private ArrayList<Mahasiswa_2511533020> daftarMahasiswa_3020;
23
24     // Constructor
25     public SortingGUI_2511533020() {
26         daftarMahasiswa_3020 = new ArrayList<>();
27         initComponents_3020();
28         setTitle("Sorting Nama Mahasiswa - NIM: 2511533020");
29         setSize(900, 700);
30         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31         setLocationRelativeTo(null);
32     }
33
34     /**
35      * Inisialisasi semua komponen GUI
36      */
37     private void initComponents_3020() {
38         setLayout(new BorderLayout());
```

```

39
40 // Panel Input (atas)
41 JPanel panelInput_3020 = new JPanel(new GridLayout(2, 4, 10, 10));
42 panelInput_3020.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));
43
44 panelInput_3020.add(new JLabel("Nama:"));
45 fieldNama_3020 = new JTextField();
46 panelInput_3020.add(fieldNama_3020);
47
48 panelInput_3020.add(new JLabel("NIM:"));
49 fieldNim_3020 = new JTextField();
50 panelInput_3020.add(fieldNim_3020);
51
52 panelInput_3020.add(new JLabel("Program Studi:"));
53 fieldProdi_3020 = new JTextField();
54 panelInput_3020.add(fieldProdi_3020);
55
56 btnTambah_3020 = new JButton("Tambah Data");
57 btnHapus_3020 = new JButton("Hapus Data");
58 panelInput_3020.add(btnTambah_3020);
59 panelInput_3020.add(btnHapus_3020);
60
61 add(panelInput_3020, BorderLayout.NORTH);
62
63 // Panel Tengah (tabel dan visualisasi)
64 JPanel panelTengah_3020 = new JPanel(new GridLayout(1, 2, 10, 10));
65
66 // Tabel data mahasiswa
67 String[] kolom_3020 = {"No", "Nama", "NIM", "Prodi"};
68 tableModel_3020 = new DefaultTableModel(kolom_3020, 0);
69 tabelMahasiswa_3020 = new JTable(tableModel_3020);
70 scrollTabel_3020 = new JScrollPane(tabelMahasiswa_3020);
71 scrollTabel_3020.setBorder(BorderFactory.createTitledBorder("Data Mahasiswa"));
72 panelTengah_3020.add(scrollTabel_3020);
73
74 // Area visualisasi sorting (GUI)
75 areaVisualisasi_3020 = new JTextArea();
76 areaVisualisasi_3020.setEditable(false);
77
78 areaVisualisasi_3020.setFont(new Font("Monospaced", Font.PLAIN, 12));
79 scrollVisual_3020 = new JScrollPane(areaVisualisasi_3020);
80 scrollVisual_3020.setBorder(BorderFactory.createTitledBorder("Visualisasi Proses Sorting (GUI)"));
81 panelTengah_3020.add(scrollVisual_3020);
82
83 add(panelTengah_3020, BorderLayout.CENTER);
84
85 // Panel Bawah (pilihan algoritma dan tombol sort)
86 JPanel panelBawah_3020 = new JPanel(new FlowLayout());
87
88 panelBawah_3020.add(new JLabel("Pilih Algoritma Sorting:"));
89 String[] algoritma_3020 = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
90 comboAlgoritma_3020 = new JComboBox<>(algoritma_3020);
91 panelBawah_3020.add(comboAlgoritma_3020);
92
93 btnSort_3020 = new JButton("Mulai Sorting");
94 panelBawah_3020.add(btnSort_3020);
95
96 add(panelBawah_3020, BorderLayout.SOUTH);
97
98 // Event Listener
99 btnTambah_3020.addActionListener(e -> tambahData_3020());
100 btnHapus_3020.addActionListener(e -> hapusData_3020());
101 btnSort_3020.addActionListener(e -> mulaiSorting_3020());
102
103 }
104
105 /**
106  * Menambahkan data mahasiswa ke dalam ArrayList dan tabel
107  */
108 private void tambahData_3020() {
109     String nama = fieldNama_3020.getText().trim();
110     String nim = fieldNim_3020.getText().trim();
111     String prodi = fieldProdi_3020.getText().trim();
112
113     if (nama.isEmpty() || nim.isEmpty() || prodi.isEmpty()) {
114         JOptionPane.showMessageDialog(this, "Semua field harus diisi!");
115         return;
116     }
117 }

```

```

115
116     Mahasiswa_2511533020 mhs = new Mahasiswa_2511533020(nama, nim, prodi);
117     daftarMahasiswa_3020.add(mhs);
118     refreshTabel_3020();
119
120     fieldNama_3020.setText("");
121     fieldNim_3020.setText("");
122     fieldProdi_3020.setText("");
123     fieldNama_3020.requestFocus();
124
125     JOptionPane.showMessageDialog(this, "Data berhasil ditambahkan!");
126 }
127
128 /**
129  * Menghapus data mahasiswa yang dipilih dari tabel
130  */
131 private void hapusData_3020() {
132     int selectedRow = tabelMahasiswa_3020.getSelectedRow();
133     if (selectedRow == -1) {
134         JOptionPane.showMessageDialog(this, "Pilih data yang akan dihapus!");
135         return;
136     }
137
138     daftarMahasiswa_3020.remove(selectedRow);
139     refreshTabel_3020();
140     JOptionPane.showMessageDialog(this, "Data berhasil dihapus!");
141 }
142
143 /**
144  * Merefresh tampilan tabel sesuai isi ArrayList
145  */
146 private void refreshTabel_3020() {
147     tableModel_3020.setRowCount(0);
148     for (int i = 0; i < daftarMahasiswa_3020.size(); i++) {
149         Mahasiswa_2511533020 m = daftarMahasiswa_3020.get(i);
150         tableModel_3020.addRow(new Object[]{
151             i + 1,
152             m.getNama_3020(),

```

```

153             m.getNim_3020(),
154             m.getProdi_3020()
155         });
156     }
157 }
158
159 /**
160  * Memulai proses sorting sesuai algoritma yang dipilih
161  * Visualisasi ditampilkan di console DAN di GUI
162  */
163 private void mulaiSorting_3020() {
164     if (daftarMahasiswa_3020.isEmpty()) {
165         JOptionPane.showMessageDialog(this, "Tidak ada data untuk di-sort!");
166         return;
167     }
168
169     String algoritma = (String) comboAlgoritma_3020.getSelectedItem();
170
171     // Bersihkan area visualisasi GUI
172     areaVisualisasi_3020.setText("");
173
174     // Tampilkan di console
175     System.out.println("\n=====");
176     System.out.println("=== " + algoritma + " ===");
177     System.out.println("=====");
178
179     // Tampilkan di GUI
180     areaVisualisasi_3020.append("=== " + algoritma + " ===\n\n");
181
182     // Tampilkan data awal (console dan GUI)
183     System.out.print("Data Awal: ");
184     areaVisualisasi_3020.append("Data Awal:\n");
185     tampilkanDataAwal_3020(true); // true = cetak ke console juga
186
187     // Buat salinan data untuk di-sort
188     ArrayList<Mahasiswa_2511533020> dataCopy = new ArrayList<>(daftarMahasiswa_3020);
189
190     if (algoritma.equals("Insertion Sort")) {

```

```

191         insertionSort_3020(dataCopy);
192     } else if (algoritma.equals("Selection Sort")) {
193         selectionSort_3020(dataCopy);
194     } else if (algoritma.equals("Bubble Sort")) {
195         bubbleSort_3020(dataCopy);
196     }
197
198     // Tampilkan hasil akhir (console dan GUI)
199     System.out.print("\n=== HASIL AKHIR ===\n");
200     areaVisualisasi_3020.append("\n\n=== HASIL AKHIR ===\n");
201     tampilkanHasilAkhir_3020(dataCopy, true);
202
203     // Update daftar mahasiswa dengan hasil sorting
204     daftarMahasiswa_3020 = dataCopy;
205     refreshTabel_3020();
206
207     JOptionPane.showMessageDialog(this, "Sorting selesai! Lihat console untuk detail langkah-langkah sorting.");
208 }
209
210 /**
211  * Menampilkan data awal ke area visualisasi (GUI) dan console
212  */
213 private void tampilkanDataAwal_3020(boolean keConsole) {
214     StringBuilder sb = new StringBuilder("");
215     for (int i = 0; i < daftarMahasiswa_3020.size(); i++) {
216         sb.append(daftarMahasiswa_3020.get(i).getNama_3020());
217         if (i < daftarMahasiswa_3020.size() - 1) {
218             sb.append(", ");
219         }
220     }
221     sb.append("\n\n");
222     areaVisualisasi_3020.append(sb.toString());
223
224     if (keConsole) {
225         System.out.print(sb.toString());
226     }
227 }
228
229
230 /**
231  * Menampilkan hasil akhir sorting ke area visualisasi (GUI) dan console
232  */
233 private void tampilkanHasilAkhir_3020(ArrayList<Mahasiswa_2511533020> data, boolean keConsole) {
234     StringBuilder sb = new StringBuilder("");
235     for (int i = 0; i < data.size(); i++) {
236         sb.append(data.get(i).getNama_3020());
237         if (i < data.size() - 1) {
238             sb.append(", ");
239         }
240     }
241     sb.append("\n\n");
242     areaVisualisasi_3020.append(sb.toString());
243
244     if (keConsole) {
245         System.out.print(sb.toString());
246     }
247 }
248
249
250 /**
251  * Menampilkan langkah sorting ke area visualisasi (GUI) dan console
252  */
253 private void tampilkanLangkahSorting_3020(ArrayList<Mahasiswa_2511533020> data, String prefix, int nomor, boolean keConsole) {
254     StringBuilder sb = new StringBuilder(prefix + " " + nomor + " : ");
255     for (int i = 0; i < data.size(); i++) {
256         sb.append(data.get(i).getNama_3020());
257         if (i < data.size() - 1) {
258             sb.append(", ");
259         }
260     }
261     sb.append("\n\n");
262     areaVisualisasi_3020.append(sb.toString());
263
264     if (keConsole) {
265         System.out.print(sb.toString());
266     }

```

```

267     }
268 }
269
270 /**
271  * INSERTION SORT - mengurutkan data berdasarkan nama (case insensitive)
272  * Menampilkan setiap langkah perubahan ke console dan GUI
273  */
274 private void insertionSort_3020(ArrayList<Mahasiswa_2511533020> data) {
275     int langkah = 1;
276     System.out.println("\n--- PROSES INSERTION SORT ---");
277     areaVisualisasi_3020.append("--- PROSES INSERTION SORT ---\n");
278
279     for (int i = 1; i < data.size(); i++) {
280         Mahasiswa_2511533020 key = data.get(i);
281         int j = i - 1;
282
283         String penjelasan = "Memasukkan \"" + key.getNama_3020() + "\" ke posisi yang tepat";
284
285         System.out.println("Langkah " + langkah + ": " + penjelasan);
286         areaVisualisasi_3020.append("Langkah " + langkah + ": " + penjelasan + "\n");
287
288         while (j >= 0 && data.get(j).getNama_3020().compareToIgnoreCase(key.getNama_3020()) > 0) {
289             data.set(j + 1, data.get(j));
290             j--;
291         }
292         data.set(j + 1, key);
293
294         tampilkanLangkahSorting_3020(data, "Hasil", langkah, true);
295         langkah++;
296     }
297 }
298
299 /**
300  * SELECTION SORT - mengurutkan data berdasarkan nama (case insensitive)
301  * Menampilkan setiap pass ke console dan GUI
302  */
303 private void selectionSort_3020(ArrayList<Mahasiswa_2511533020> data) {
304     int pass = 1;
305     System.out.println("\n--- PROSES SELECTION SORT ---");
306     areaVisualisasi_3020.append("--- PROSES SELECTION SORT ---\n");
307
308     for (int i = 0; i < data.size() - 1; i++) {
309         int minIdx = i;
310         for (int j = i + 1; j < data.size(); j++) {
311             if (data.get(j).getNama_3020().compareToIgnoreCase(data.get(minIdx).getNama_3020()) < 0) {
312                 minIdx = j;
313             }
314         }
315
316         String penjelasan = "Menukar \"" + data.get(minIdx).getNama_3020() + "\" dengan \"" + data.get(i).getNama_3020() + "\"";
317
318         Mahasiswa_2511533020 temp = data.get(minIdx);
319         data.set(minIdx, data.get(i));
320         data.set(i, temp);
321
322         System.out.println("Pass " + pass + ": " + penjelasan);
323         areaVisualisasi_3020.append("Pass " + pass + ": " + penjelasan + "\n");
324
325         tampilkanLangkahSorting_3020(data, "Hasil", pass, true);
326         pass++;
327     }
328 }
329
330 /**
331  * BUBBLE SORT - mengurutkan data berdasarkan nama (case insensitive)
332  * Menampilkan setiap pass ke console dan GUI
333  */
334 private void bubbleSort_3020(ArrayList<Mahasiswa_2511533020> data) {
335     int pass = 1;
336     System.out.println("\n--- PROSES BUBBLE SORT ---");
337     areaVisualisasi_3020.append("--- PROSES BUBBLE SORT ---\n");
338
339     for (int i = 0; i < data.size() - 1; i++) {
340         boolean swapped = false;
341         String pertukaranTerakhir = "";
342

```

```

343     for (int j = 0; j < data.size() - i - 1; j++) {
344         if (data.get(j).getNama_3020().compareToIgnoreCase(data.get(j + 1).getNama_3020()) > 0) {
345             Mahasiswa_2511533020 temp = data.get(j);
346             data.set(j, data.get(j + 1));
347             data.set(j + 1, temp);
348             swapped = true;
349             pertukaranTerakhir = "Menukar \"" + data.get(j).getNama_3020() + "\" dengan \"" + data.get(j + 1).getNama_3020() + "\"";
350         }
351     }
352
353
354     String penjelasan;
355     if (swapped) {
356         penjelasan = pertukaranTerakhir + " (menggelembungkan nilai terbesar)";
357     } else {
358         penjelasan = "Tidak ada pertukaran, array sudah terurut";
359     }
360
361     System.out.println("Pass " + pass + ": " + penjelasan);
362     areaVisualisasi_3020.append("Pass " + pass + ": " + penjelasan + "\n");
363
364
365     tampilkanLangkahSorting_3020(data, "Hasil", pass, true);
366     pass++;
367
368     if (!swapped) {
369         break;
370     }
371 }
372 }
373
374 /**
375  * Main method - menjalankan program GUI
376  */
377 public static void main(String[] args) {
378     System.out.println("=== PROGRAM SORTING NAMA MAHASISWA ===");
379     System.out.println("NIM: 2511533020");
380     System.out.println("Visualisasi langkah-langkah sorting akan ditampilkan di console ini.\n");
381
382     SwingUtilities.invokeLater(() -> {
383         new SortingGUI_2511533020().setVisible(true);
384     });
385 }
386 }

```

Program ini merupakan kelas utama yang memuat GUI dan logika sorting. GUI dibangun menggunakan JFrame dengan komponen JTextField untuk input nama, nim, dan prodi; JButton untuk tombol tambah, hapus, dan mulai sorting; JComboBox untuk memilih algoritma (Insertion Sort, Selection Sort, Bubble Sort); JTable untuk menampilkan daftar mahasiswa; serta JTextArea untuk visualisasi sorting di GUI. Data mahasiswa disimpan dalam ArrayList<Mahasiswa\_2511533020>.

Method tambahData\_3020() dan hapusData\_3020() digunakan untuk mengelola data, sedangkan refreshTabel\_3020() untuk memperbarui tampilan tabel. Method mulaiSorting\_3020() menjalankan algoritma sorting yang dipilih. Ketiga algoritma (insertionSort\_3020(), selectionSort\_3020(), bubbleSort\_3020()) menggunakan compareToIgnoreCase() untuk membandingkan nama mahasiswa. Setiap langkah sorting ditampilkan di console dan GUI melalui method bantuan tampilkanDataAwal\_3020(), tampilkanHasilAkhir\_3020(), dan tampilkanLangkahSorting\_3020().